

TP N° 1: Pruebas Unitarias, Integración y Regresión



Departamento: Desarrollo Tecnológico y Productivo

Carrera: Licenciatura en Sistemas

Materia: Prueba de Software

Fecha: 2do Cuatrimestre del año 2025

Profesores: Eduardo Diez , Pablo Diego San Roman

Estudiantes:

- Moliterno Gonzalez, Luciano Gabriel DNI 40238958
- Sergio Rodrigo Cadima Villarroel DNI 41546475
- Ruiz Pereira, Roberto Andres DNI 37762826

Para comenzar decidimos hacer las pruebas sobre un proyecto que ya tenemos creado, este es la lógica de negocios de un supermercado, en base a este código decidimos realizar las pruebas unitarias y de integración

Capturas de test unitarios

AdminClienteTest

The screenshot shows the AdminClienteTest class in the Package Explorer and the Console window. The class is located in the package testUnitarios and contains the following code:

```

1 package testUnitarios;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 public class AdminClienteTest {
6
7     private AdminCliente adminCliente;
8
9     @BeforeEach
10    public void setUp() {
11        adminCliente = new AdminCliente();
12    }
13
14    @Test
15    public void testAgregarCliente() throws Exception {
16
17        boolean agregado = adminCliente.agregarCliente("Roberto A", 12345678L, "9 de julio 336");
18        assertTrue(agregado, "El cliente debería agregarse correctamente");
19        assertEquals(1, adminCliente.getListCliente().size(), "La lista debe tener 1 cliente");
20    }
21
22    @Test
23    public void testAgregarClienteDuplicado() throws Exception {
24
25        adminCliente.agregarCliente("roberto ermes", 12345678L, "25 de mayo 369");
26        Exception exception = assertThrows(Exception.class, () -> {
27            adminCliente.agregarCliente("Pedro g", 12345678L, "3 de mayo456");
28        });
29    }
30
31 }

```

The Console window shows the test results for the AdminClienteTest class, indicating that all tests passed successfully.

Element	Coverage	Covered Instructio...	Missed Instructions	Total Instructions
Supermercado	14.1 %	503	3,052	3,555

En esta prueba unitaria se testeo la clase adminCliente, es ella se probó:

- AgregarCliente
- AgregarClienteDuplicado
- TraerClientePorDni
- TraerClientePorId
- ToString

sin ningún fallo al ser ejecutados.

AdmProductoTest

The screenshot shows the AdmProductoTest class in the Package Explorer and the Console window. The class is located in the package testUnitarios and contains the following code:

```

1 package testUnitarios;
2
3 import modelo.AdmProducto;
4
5 public class AdmProductoTest {
6
7     private AdmProducto admin;
8
9     @BeforeEach
10    public void setUp() throws Exception {
11        admin = new AdmProducto();
12        Producto p1 = mock(Producto.class);
13        Producto p2 = mock(Producto.class);
14        when(p1.getIdProducto()).thenReturn(1);
15        when(p1.getNombre()).thenReturn("Pan");
16        when(p1.getPrecio()).thenReturn(150.0f);
17        when(p2.getIdProducto()).thenReturn(2);
18        when(p2.getNombre()).thenReturn("Leche");
19        when(p2.getPrecio()).thenReturn(300.0f);
20        admin.getListProducto().add(p1);
21        admin.getListProducto().add(p2);
22    }
23
24    @Test
25    public void testAgregarProductoNuevo() throws Exception {
26
27    }
28
29 }

```

The Console window shows the test results for the AdmProductoTest class, indicating that all tests passed successfully.

Element	Coverage	Covered Instruct
modelo	6.3 %	
testUnitarios	19.7 %	
TestSupermercado.java	0.0 %	
TestFunciones.java	0.0 %	
CarritoTest.java	0.0 %	
AdminClienteTest.java	0.0 %	
AdmProductoTest.java	92.5 %	

En esta prueba unitaria se testeo la clase **AdmProducto**, es ella se probó:

- AgregarProductoNuevo
- AgregarProductoDuplicado
- TraerProductoPorNombreExistente
- TraerProductoPorNombreNoExistente
- TraerProductoPorIdExistente

CarritoTest

The screenshot shows the CarritoTest class in an IDE. The left pane displays the test results for CarritoTest, showing four tests: testAgregarItem(), testFallaCalculoTotal(), testCalcularTotal(), and testEliminarItem(). The testFallaCalculoTotal() test is highlighted, indicating a failure. The right pane shows the source code of CarritoTest, with the testFallaCalculoTotal() method highlighted. The failure trace in the bottom pane shows the error: org.opentest4j.AssertionFailedError: El total no coincide con el esperado.

```
44 @Test
45 void testEliminarItem() throws Exception {
46     carrito.agregarItem(producto1, 2);
47     carrito.eliminarItem(producto1, 1);
48     assertEquals(1, carrito.trasItemCarrito(producto1).getCantidad());
49 }
50
51 @Test
52 void testFallaCalculoTotal() {
53     carrito.agregarItem(producto1, 1);
54     carrito.agregarItem(producto2, 1);
55     float total = carrito.calcularTotal();
56
57     assertEquals(5000f, total, "El total no coincide con el esperado");
58 }
59 }
60
```

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Supermercado	10,3 %	347	3.015	3.362

En esta prueba unitaria se testeo la clase **Carrito**, es ella se probó:

- AgregarItem
- CalcularTotal
- EliminarItem
- FallaCalculoTotal

sin ninguna falla.

TestFunciones

The screenshot shows the TestFunciones class in an IDE. The left pane displays the test results for TestFunciones, showing 13 tests: testTraerFechaCorta(), testTraerFechaLarga(), testAproximar2Decimal(), testTraerHoraCorta(), testEsNumeroYLetra(), testEsDiaHabil(), testTraerDiaDeLaSemana(), testConvertirADouble(), testEsBisiesto(), testTraerMesEnLetras(), testEsCadenaNrosYLetras(), testEsFechaValida(), and testTraerFechaValida(). The testEsFechaValida() test is highlighted, indicating a failure. The right pane shows the source code of TestFunciones, with the testEsFechaValida() method highlighted. The failure trace in the bottom pane shows the error: org.opentest4j.AssertionFailedError: expected: <14:30> but was: <14:30:00>.

```
1 package testUnitarios;
2
3 import modelo.Funciones;
4
5 public class TestFunciones {
6
7     @Test
8     public void testEsBisiesto() {
9         assertTrue(Funciones.esBisiesto(2024));
10        assertFalse(Funciones.esBisiesto(2023));
11    }
12
13     @Test
14     public void testEsFechaValida() {
15         assertTrue(Funciones.esFechaValida(LocalDate.of(2024, 2, 29))); // Año bisiesto
16         assertFalse(Funciones.esFechaValida(LocalDate.of(2023, 2, 29))); // No bisiesto
17         assertTrue(Funciones.esFechaValida(LocalDate.of(2023, 12, 31)));
18    }
19 }
20
```

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Supermercado	14,3 %	482	2.880	3.362

En esta prueba unitaria se testeo la clase **Funciones**, es ella se probó:

- esBisiesto
- esFechaValida
- TraerFechaCorta

- TraerHoraCorta
- EsDiaHabil
- TraerDiaDeLaSemana
- TraerMesEnLetras
- TraerFechaLarga
- TraerCantDiasDeUnMes
- Aproximar2Decimal
- EsNumeroYLetra
- EsCadenaNrosYLetras
- ConvertirADouble

De todas las funciones la única que falla es la función esFechaValida porque se ingrese una fecha que no existe en un año bisiesto

TestSupermercado

Finished after 4.209 seconds

Runs: 7/7 Errors: 0 Failures: 1

TestSupermercado [Runner: JUnit 5] (3.872 s)

- testAgregarCarrito() (3.723 s)
- testAgregarCliente() (0.015 s)
- testEliminarClienteConCarritoAsociadoLanzaExcepcion() (0.015 s)
- testEliminarProductoNoExistenteLanzaExcepcion() (0.015 s)
- testFallaIntencionalmente() (0.029 s)
- testAgregarProducto() (0.008 s)
- testCalcularTotalPorDniCliente() (0.038 s)

```

98
99
100 @Test
101 public void testCalcularTotalPorDniCliente() throws Exception {
102     when(clienteMock.getDni()).thenReturn(12345678L);
103     when(admClienteMock.traerCliente(12345678L)).thenReturn(clienteMock);
104     Carrito carrito1 = mock(Carrito.class);
105     Carrito carrito2 = mock(Carrito.class);
106     when(carrito1.getClient()).thenReturn(clienteMock);
107     when(carrito2.getClient()).thenReturn(clienteMock);
108     when(carrito1.calcularTotal()).thenReturn(200f);
109     when(carrito2.calcularTotal()).thenReturn(300f);
110     when(admCarritoMock.getIstCarrito()).thenReturn(java.util.List.of(carrito1, carrito2));
111     float total = supermercado.calcularTotal(12345678L);
112     assertEquals(500f, total, 0.01, "El total calculado debería ser 500");
113 }
114
115
116 @Test
117 public void testFallaIntencionalmente() throws Exception {
118     when(admProductoMock.agregarProducto("Yerba", 1500f)).thenReturn(true);
119     boolean agregado = supermercado.agregarProducto("Yerba", 1500f);
120     assertFalse(agregado, "Este test está diseñado para fallar intencionalmente");
121 }
122

```

Console Coverage X

TestSupermercado (9 oct. 2025 17:34:53)

Element	Coverage	Covered Instructio...	Missed Instructions	Total Instructions
> Supermercado	14.1 %	503	3,052	3,555

En esta prueba unitaria se testeo la clase **Supermercado**, se crearon varios clase mocks para hacer el test, las funciones que se probaron fueron:

- AgregarProducto
- EliminarProductoNoExistenteLanzaExcepcion
- AgregarCliente
- EliminarClienteConCarritoAsociadoLanzaExcepcion
- AgregarCarrito
- CalcularTotalPorDniCliente
- fallaIntencionalmente

De todas las funciones la única que falla es la función fallaIntencionalmente porque se hizo a proposito para generar una falla

TEST DE INTEGRACION

FuncionesTest

Finished after 0,19 seconds

Runs: 15/15 Errors: 1 Failures: 0

FuncionesTest [Runner: JUnit 5] (0,035 s)

- testEsCadenaLetras() (0,000 s)
- testEsLetra() (0,000 s)
- testTraerFechaCorta() (0,008 s)
- testTraerFechaLarga() (0,006 s)
- testAproximar2Decimal() (0,001 s)
- testTraerHoraCorta() (0,003 s)
- testEsDiaHabil() (0,001 s)
- testTraerDiaDeLaSemana() (0,001 s)
- testConvertirADouble() (0,001 s)
- testEsBisiesto() (0,001 s)
- testTraerMesEnLetras() (0,001 s)
- testEsFechaValida() (0,007 s)
- testTraerCantDiasDeUnMes() (0,001 s)
- testEsNumero() (0,000 s)
- testEsCadenaNros() (0,001 s)

Failure Trace

```
java.time.DateTimeException: Invalid date 'February 29'
    at java.base/java.time.LocalDate.create(LocalDate.java:275)
    at java.base/java.time.LocalDate.of(LocalDate.java:275)
    at testsIntegracion.FuncionesTest.testEsFechaValida(FuncionesTest.java:21)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1541)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1541)
```

```
1 package testsIntegracion;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5
6
7
8
9
10 class FuncionesTest {
11
12     @Test
13     void testEsBisiesto() {
14         assertTrue(Funciones.esBisiesto(2024));
15         assertFalse(Funciones.esBisiesto(2023));
16     }
17
18     @Test
19     void testEsFechaValida() {
20         assertTrue(Funciones.esFechaValida(LocalDate.of(2024, 2, 29)));
21         assertFalse(Funciones.esFechaValida(LocalDate.of(2023, 2, 29)));
22     }
23 }
```

Element	Coverage	Covered Instructions	Missed Instru
Supermercado	13,1 %	439	

En esta prueba de integración se testeó la clase **Funciones**, la única función que falla es `testFechaValida` porque se definió una fecha inexistente en ese año.

Instalación y uso de Mockito en Eclipse

Al desarrollar aplicaciones Java, es fundamental garantizar la calidad del código mediante pruebas unitarias. Estas permiten comprobar que los métodos de cada clase funcionan correctamente de manera independiente.

Sin embargo, muchas veces las clases dependen de otras, lo que dificulta probarlas de forma aislada. En estos casos se utilizan mocks y stubs, que son objetos simulados que reemplazan temporalmente a las dependencias reales.

Mockito es una librería de Java que permite crear estos objetos simulados de manera sencilla. Con ella, es posible imitar el comportamiento de clases reales y definir las respuestas que devolverán sus métodos, sin ejecutar la lógica interna de las mismas. En este informe se explica cómo instalar Mockito en Eclipse sin utilizar frameworks como Maven, y cómo usarlo en un ejemplo práctico de pruebas unitarias con JUnit 5.

Instalación manual de Mockito (.jar) en Eclipse

Descargar la librería

Para instalar Mockito manualmente en un proyecto Java: Accediendo al repositorio oficial de dependencias: <https://mvnrepository.com/artifact/org.mockito/mockito-core>

Hice clic en la versión estable más reciente (en mi caso, la 5.20.0).



Mockito Core
Mockito mock objects library core API and implementation

License	MIT
Categories	Mocking
Tags	quality mock mocking testing mockito
HomePage	https://github.com/mockito/mockito
Ranking	#8 in MvnRepository (See Top Artifacts) #1 in Mocking
Used By	36,260 artifacts

Central (346) Appodeal (18) Novoda (1) ICM (3) BT Mockito (251)

	Version ▼	Vulnerabilities	Repository	Usages	Date
5.20.x	5.20.0		Central	327	Sep 20, 2025
5.19.x	5.19.0		Central	881	Aug 15, 2025
5.18.x	5.18.0		Central	1,796	May 20, 2025
5.17.x	5.17.0		Central	1,567	Apr 04, 2025
	5.16.1		Central	769	Mar 15, 2025

Descargué el archivo mockito-core-5.20.0.jar.

Home » org.mockito » mockito-core » 5.20.0



Mockito Core » 5.20.0
Mockito mock objects library core API and implementation

License	MIT
Categories	Mocking
Tags	quality mock mocking testing mockito
HomePage	https://github.com/mockito/mockito
Date	Sep 20, 2025
Files	pom (2 KB) jar (693 KB) View All
Repositories	Central
Ranking	#8 in MvnRepository (See Top Artifacts) #1 in Mocking
Used By	36,260 artifacts

Maven Gradle SBT Mill Ivy Grape Leiningen Buildr

Scope: Test ▼

<!-- https://mvnrepository.com/artifact/org.mockito/mockito-core -->

También descargamos los stubs Mockito, elegimos las versiones más actuales

Home » net.bytebuddy » byte-buddy » 1.17.8



Byte Buddy (without Dependencies) » 1.17.8

Byte Buddy is a Java library for creating Java classes at run time. This artifact is a build of Byte Buddy with all ASM dependencies repackaged into its own name space.

License	Apache 2.0
Categories	Bytecode Libraries
Tags	jvm asm bytecode
Date	Oct 09, 2025
Files	pom (18 KB) jar (8.6 MB) View All
Repositories	Central
Ranking	#198 in MvnRepository (See Top Artifacts) #3 in Bytecode Libraries
Used By	2,920 artifacts

[Maven](#) [Gradle](#) [SBT](#) [Mill](#) [Ivy](#) [Grape](#) [Leiningen](#) [Buildr](#)

Home » net.bytebuddy » byte-buddy-agent » 1.17.8



Byte Buddy Agent » 1.17.8

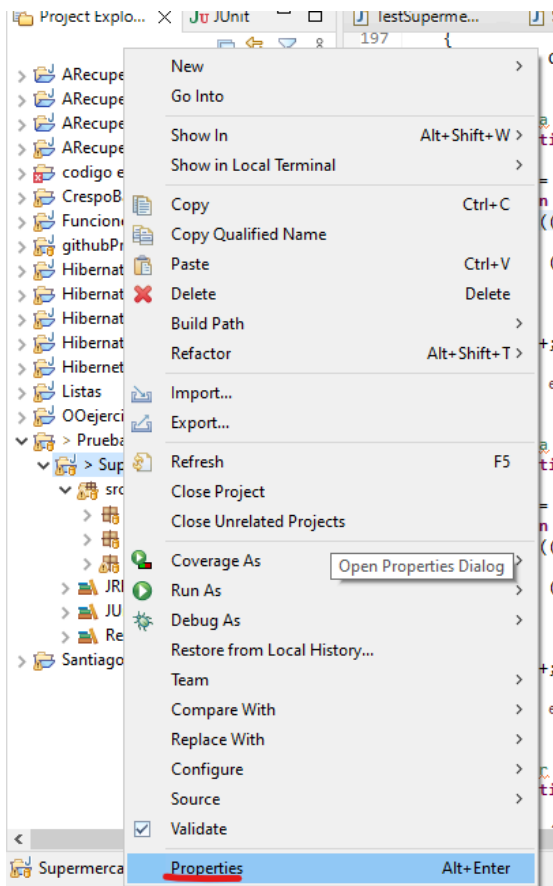
The Byte Buddy agent offers convenience for attaching an agent to the local or a remote VM.

License	Apache 2.0
Categories	Bytecode Libraries
Tags	jvm agent asm bytecode
Date	Oct 08, 2025
Files	pom (14 KB) jar (357 KB) View All
Repositories	Central
Ranking	#658 in MvnRepository (See Top Artifacts) #10 in Bytecode Libraries
Used By	941 artifacts

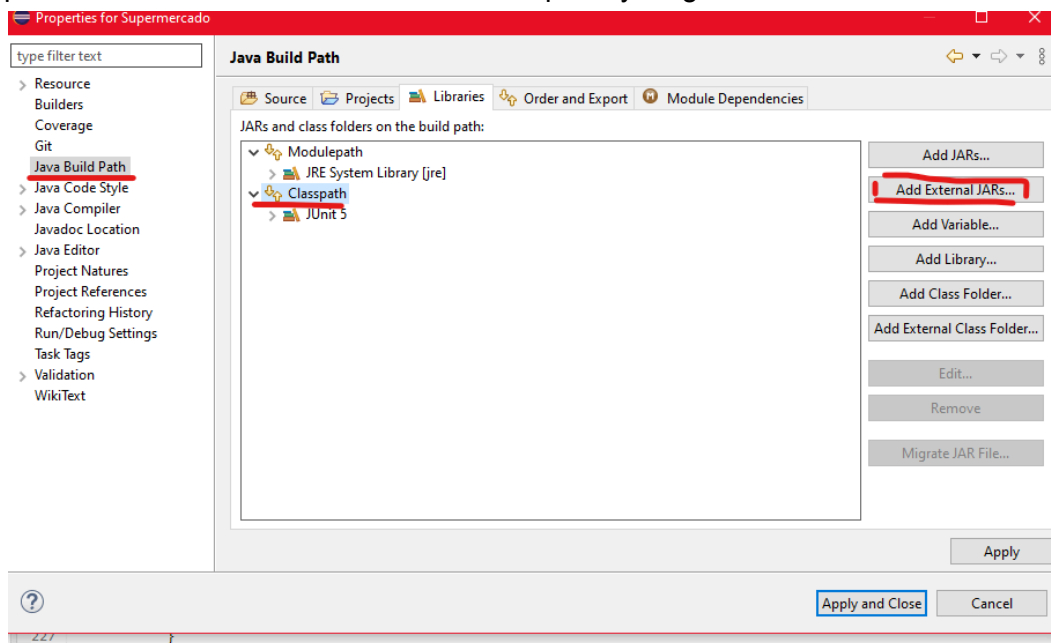
[Maven](#) [Gradle](#) [SBT](#) [Mill](#) [Ivy](#) [Grape](#) [Leiningen](#) [Buildr](#)

Agregar los archivos .jar al proyecto

Abrí Eclipse y vamos a mi proyecto Java.
Hice clic derecho sobre el nombre del proyecto → Properties



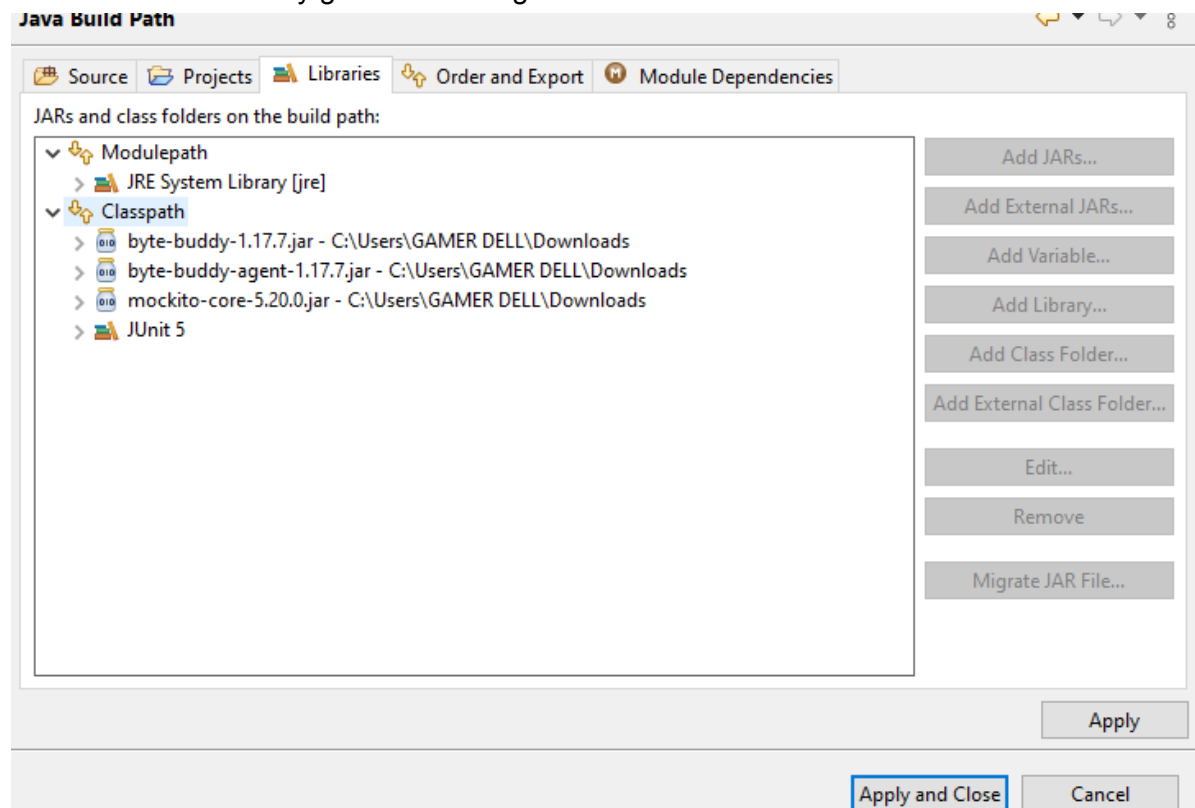
Se abrirá esta ventana, en ella tenemos que darle click al “Java build Path”, abrimos la pestaña de “Libraries ” seleccionar “Classpath” y luego “Add External JARs ...”



Buscamos y introducimos el archivo mockito-core-5.20.0.jar y los stubs Mockito al proyecto

> Este equipo > Descargas >		▼ ↻		Buscar en ... 🔍
	Nombre	Fecha de modificación	Tipo	Ta
onal	 byte-buddy-1.17.7	04/10/2025 01:40 p. m.	Executable Jar File	
	 byte-buddy-agent-1.17.7	04/10/2025 01:39 p. m.	Executable Jar File	
	 mockito-core-5.20.0	04/10/2025 01:39 p. m.	Executable Jar File	
	 TP1	29/09/2025 03:52 p. m.	Microsoft Edge P...	

Confirmé los cambios y guardé la configuración.



De esta forma, Mockito y JUnit 5 quedaron listos para usar en el proyecto, sin necesidad de un gestor de dependencias como Maven o Gradle.

Uso de Mockito en pruebas unitarias

Conceptos básicos

Mock: es un objeto falso que replica el comportamiento de una clase real.
Se utiliza para probar una clase sin depender de implementaciones externas.

Stub: es un tipo de mock más simple, que devuelve valores preestablecidos.

El propósito de usar mocks es desacoplar la lógica de negocio y asegurarnos de que los métodos se ejecuten correctamente, sin importar si las dependencias reales están finalizadas o no.

Creación de un mock

Los mocks se crean llamando al método:

```
Producto p1 = mock(Producto.class);
```

Esto genera un objeto simulado de la clase Producto, el cual puede configurarse para devolver datos específicos cuando sea llamado.

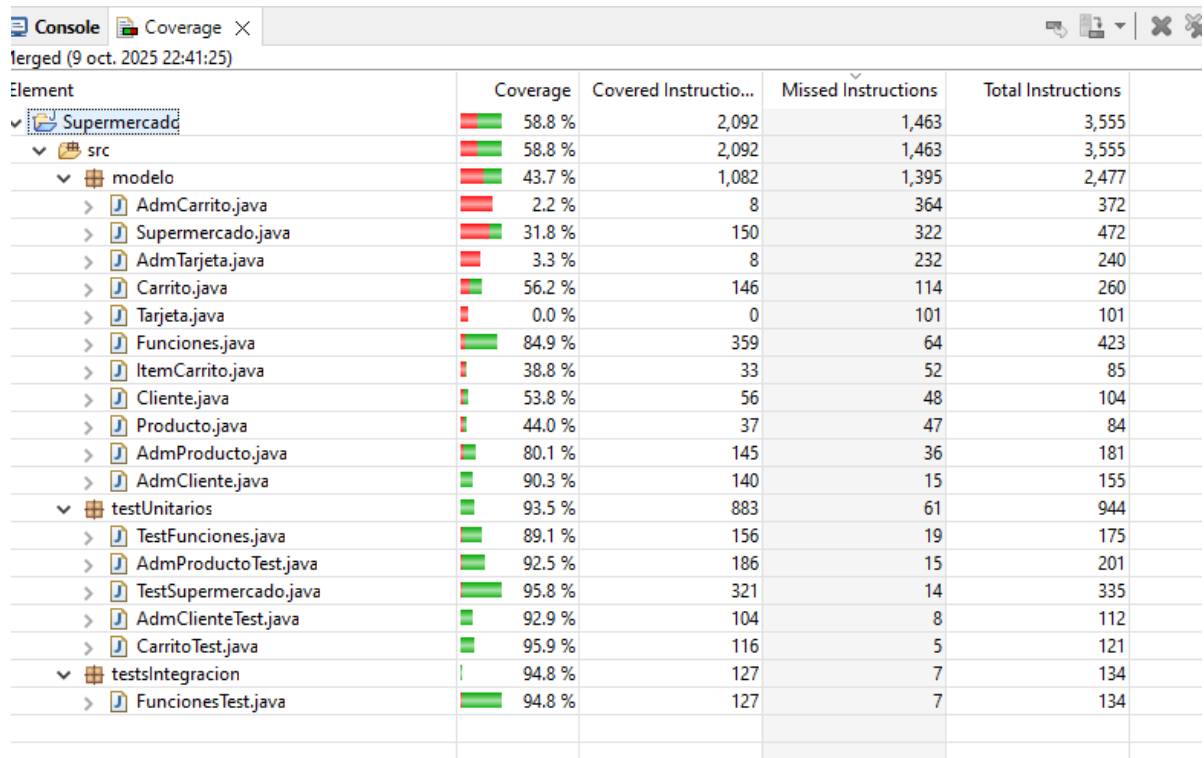
Configuración del comportamiento simulado

Para definir qué debe devolver un mock ante ciertos llamados, se utiliza el método `when(...).thenReturn(...)`:

```
when(p1.getNombre()).thenReturn("Pan");  
when(p1.getPrecio()).thenReturn(150.0f);
```

De esta manera, al invocar `p1.getNombre()`, el mock devolverá "Pan" sin ejecutar ningún código real.

COBERTURAS



Element	Coverage	Covered Instructio...	Missed Instructions	Total Instructions
✓ Supermercado	58.8 %	2,092	1,463	3,555
▼ src	58.8 %	2,092	1,463	3,555
▼ modelo	43.7 %	1,082	1,395	2,477
> AdmCarrito.java	2.2 %	8	364	372
> Supermercado.java	31.8 %	150	322	472
> AdmTarjeta.java	3.3 %	8	232	240
> Carrito.java	56.2 %	146	114	260
> Tarjeta.java	0.0 %	0	101	101
> Funciones.java	84.9 %	359	64	423
> ItemCarrito.java	38.8 %	33	52	85
> Cliente.java	53.8 %	56	48	104
> Producto.java	44.0 %	37	47	84
> AdmProducto.java	80.1 %	145	36	181
> AdmCliente.java	90.3 %	140	15	155
▼ testUnitarios	93.5 %	883	61	944
> TestFunciones.java	89.1 %	156	19	175
> AdmProductoTest.java	92.5 %	186	15	201
> TestSupermercado.java	95.8 %	321	14	335
> AdmClienteTest.java	92.9 %	104	8	112
> CarritoTest.java	95.9 %	116	5	121
▼ testsIntegracion	94.8 %	127	7	134
> FuncionesTest.java	94.8 %	127	7	134

Esta imagen nos indica los porcentajes de cobertura que tienen nuestras pruebas unitarias y de integración en relación a las clases del supermercado

Conclusión

En este trabajo se instaló y configuró manualmente la librería Mockito en un proyecto Java de Eclipse, utilizando archivos .jar sin frameworks externos.

Gracias a Mockito fue posible crear objetos simulados de la clase Producto, lo que permitió probar los métodos de AdmProducto, por ejemplo, de forma aislada y controlada.

Esta metodología mejora considerablemente la calidad del código y permite detectar errores lógicos de manera temprana durante el desarrollo.

REPOSITORIO DE GITHUB

<https://github.com/rpruiz33/TP-prueba-de-SW.git>